Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

Capstone Project Report
Project Title : Online Examination & Assessment Platform

Problem Statement - Online Examination & Assessment Platform

Objective:

Develop a complete enterprise-grade application using layered architecture with PostgreSQL, ASP.NET Core Web API, and Angular frontend.

Core Modules:

- Question Bank
- Exam Scheduling
- Auto Evaluation
- Certificates

Key Features:

- Timed exams
- Analytics dashboard
- Real-time notifications

Suggested Architecture:

Angular Frontend → ASP.NET Core Web API → Service Layer → Repository Layer → PostgreSQL Database

Azure Integration:

- Azure Notification Hub

Student Deliverables:

- Requirement Document
- ER Diagram
- API Documentation
- Angular Screens
- Authentication & Authorization
- CRUD APIs
- Search & Pagination
- Deployment Steps

Phase 2 Enhancements:

- Scalable Kubernetes deployment

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

I. Phase 1 : Understanding problem statement and requirement analysis

- The problem statement revolves around the idea of users taking an online timed exam through the application which composes of a dashboard consisting of the details of the user and their activity i.e.; how many exams have they taken so far and for every exam they've taken how much time, the score for every question and the right option for every wrong answer they've chosen. Other exams they can take if they want to and so on. Users upon achieving pass marks in a certain exam should also receive certificates for that particular exam taken. Users can take the test only within the time if they exceed then auto-submit the answers for which they have marked already. Upon completion they should have an analytics dashboard with all the stats wrt how they've taken the test. Real time notifications should also be incorporated for example, if the grades are out the user must be notified.
- The examiner of the system is responsible for the academic content: creating time-bounded exams, adding MCQ questions, marking the correct option for each question, assigning marks, and scheduling exams. The examiner manages the exam's details through CRUD operations and has full rights to modify exams they have scheduled. Once an exam is completed, the examiner publishes the results, which issues certificates to users who passed and triggers notifications to all candidates who attempted.
- The admin of the entire system manages the platform: user accounts (including examiner accounts), exam categories, oversight of all exams across the system, viewing every user's activity, deactivating users, and revoking certificates if needed. The admin does not create exam content but governs the platform on which examiners and users operate.
- The frontend should give options to login as a user/examiner /admin.

Actors

1. **Admin** - platform-level control: manages users and examiners, manages exam categories, views all activity across the system, can deactivate users and revoke certificates.
2. **Examiner** - academic-content control: creates and manages exams, MCQ questions, schedules, marks, publishes exams, and publishes results.
3. **User (Candidate)** - registers, logs in, browses exams (filterable by category), attempts them, views results and analytics, receives certificates and email notifications.

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

Requirement Analysis

Functional Requirement	Non-Functional Requirement
User / Examiner / Admin Registration and Login	Secure authentication using JWT
Role-based Login (Admin / Examiner / User)	Passwords stored using hashing and salting
Exam Category CRUD Operations (Admin)	Fast loading and response time
Create MCQ Questions (Examiner)	Maintainable layered architecture
Add Marks and Mark Correct Options (Examiner)	Scalable for multiple concurrent users
Edit / Delete Questions (Examiner)	Reliable auto-save functionality
Create Exams with Schedule (Examiner)	Data integrity during evaluation
Add Questions to Exams (Examiner)	Proper authorization and access control
Publish Exams for Users (Examiner)	Server-side timer enforcement
Publish Results (Examiner)	High system reliability during submissions
Browse Available Exams (User)	Fair exam attempt handling
Search / Filter Exams by Category (User)	Atomic result publishing process
View Exam Details (User)	Smooth and user-friendly UI
Start Timed Exams (User)	Optimized filtering and querying
Multiple Exam Attempts Allowed	Responsive dashboard experience
Best Score Calculation	Accurate server-controlled timing
Auto-save User Answers	Consistent attempt tracking
Manual Exam Submission	Concurrent attempt handling
Auto-submit on Timeout	Crash recovery and persistence
Automatic Evaluation of MCQ Answers	Stable API communication
Result and Score Generation	Reliable timeout handling
Analytics Dashboard (User)	Consistent evaluation logic
Per-question Result Breakdown	Secure result visibility
Certificate Generation on Pass	Efficient analytics retrieval
Certificate Download / View	Structured and readable reports
Certificate Revocation (Admin)	Permanent certificate storage
Email Notification Sending	Auditability of issued certificates
Notification History Tracking	Historical record maintenance
View All User Activities (Admin)	Background email processing
Manage Examiner Accounts (Admin)	Persistent notification storage
User Deactivation by Admin	Secure admin-only access
Publish Result Notifications	Prevent unauthorized access after deactivation
View User Performance Analytics	Reliable dashboard data consistency
View Exam Attempt History	Efficient database querying
View Correct Answers for Wrong Responses	Data persistence and recovery
Track Time Taken per Exam	Accurate time synchronization
Schedule Time-bounded Exams	Reliable scheduling mechanism

Store Attempted Exam Statistics	Maintainability and modularity
Manage Platform-wide Oversight	Proper separation of concerns

Phase 2 - Entities-Relationship Designing

I. Master Tables

1. User

- user_id – int; pk
- username – varchar; unique
- email – varchar; unique
- password_hash – varchar
- password_salt – varchar
- full_name – varchar
- role – varchar; -> Admin / Examiner / Candidate
- date_of_joining – timestamp
- is_active – bool; true or false

2. ExamCategory

- cat_id – int; pk
- cat_name – varchar; unique
- description – text; nullable

3. Question

- question_id – int; pk
- examiner_id – int; fk -> User (only users with role = Examiner)
- cat_id – int; fk -> ExamCategory
- question_text – text
- marks – decimal
- difficulty – varchar; -> Easy / Medium / Hard
- created_at – timestamp
- is_active – bool; true or false (soft delete)

4. QuestionOption

- option_id – int; pk
- question_id – int; fk -> Question
- option_text – varchar
- is_correct – bool; exactly one true per question

II. Transactional Tables

1. Exam

- exam_id – int; pk
- examiner_id – int; fk -> User (creator)
- cat_id – int; fk -> ExamCategory
- title – varchar
- instructions – text; nullable
- duration_minutes – int
- total_marks – decimal
- passing_marks – decimal
- start_time – timestamp
- end_time – timestamp
- exam_status – varchar; -> Draft / Published / ResultsPublished / Archived
- created_at – timestamp

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

2. ExamQuestion

Mapping table: which questions belong to which exam.

- exam_question_id – int; pk
- exam_id – int; fk -> Exam
- question_id – int; fk -> Question
- question_order – int (display order within the exam)
- unique (exam_id, question_id) — same question can't be added twice to one exam

3. Attempt

One row per attempt by a user on an exam. Multiple rows allowed per (user, exam) since unlimited attempts.

- attempt_id – int; pk
- user_id – int; fk -> User
- exam_id – int; fk -> Exam
- started_at – timestamp submitted_at – timestamp; nullable (null while in progress)
- attempt_status – varchar; -> InProgress / Submitted / AutoSubmitted
- score – decimal; nullable (filled on submit)
- is_passed – bool; nullable (filled on submit)
- time_taken_seconds – int; nullable

4. AttemptAnswer

One row per question per attempt. Stores the user's selected option.

- answer_id – int; pk
- attempt_id – int; fk -> Attempt
- question_id – int; fk -> Question
- selected_option_id – int; fk -> QuestionOption; nullable (null if skipped)
- is_correct – bool; nullable (filled on evaluation)
- marks_awarded – decimal; nullable (filled on evaluation)
- answered_at – timestamp
- unique (attempt_id, question_id) — one answer row per question per attempt

5. Certificate

- certificate_id – int; pk
- certificate_code – varchar; unique (human-readable, e.g., CERT-2026-000042)
- user_id – int; fk -> User
- exam_id – int; fk -> Exam
- attempt_id – int; fk -> Attempt (the attempt that earned it — best-scoring passing attempt)
- score – decimal (snapshot at issuance)
- issued_at – timestamp
- cert_status – varchar; -> Issued / Revoked
- revoked_at – timestamp; nullable
- unique (user_id, exam_id) — one certificate per user per exam

6. Notification

- notification_id – int; pk
- user_id – int; fk -> User (recipient)
- notif_type – varchar; -> ResultPublished /

CertificateIssued / CertificateRevoked
- message – text

III. Relationship Mapping

1. User → Question (Examiner authoring)

- One examiner -> Multiple questions (1-N)
- Question.examiner_id → User.user_id

2. ExamCategory → Question

- One category -> Multiple questions (1-N)
- Question.cat_id → ExamCategory.cat_id

3. Question → QuestionOption

- One question -> Multiple options (1-N)
- QuestionOption.question_id → Question.question_id

4. User → Exam (Examiner creating)

- One examiner -> Multiple exams (1-N)
- Exam.examiner_id → User.user_id

5. ExamCategory → Exam

- One category -> Multiple exams (1-N)
- Exam.cat_id → ExamCategory.cat_id
- related_exam_id – int; fk -> Exam; nullable
- is_read – bool; default false
- created_at – timestamp

6. Exam ↔ Question (via ExamQuestion)

- Many exams -> Many questions (N-N)
- Resolved through the ExamQuestion mapping table
- ExamQuestion.exam_id → Exam.exam_id
- ExamQuestion.question_id → Question.question_id

7. User → Attempt (Candidate attempting)

- One user -> Multiple attempts (1-N)
- Attempt.user_id → User.user_id

8. Exam → Attempt

- One exam -> Multiple attempts (1-N)
- Attempt.exam_id → Exam.exam_id

9. Attempt → AttemptAnswer

- One attempt -> Multiple answers (1-N), one per question
- AttemptAnswer.attempt_id → Attempt.attempt_id

10. Question → AttemptAnswer

- One question -> Multiple answer records across attempts (1-N)
- AttemptAnswer.question_id → Question.question_id

11. QuestionOption → AttemptAnswer

- One option -> Multiple answer records (1-N, since many users may pick it)
- AttemptAnswer.selected_option_id → QuestionOption.option_id

12. User → Certificate

- One user -> Multiple certificates (1-N, one per exam they pass)
- Certificate.user_id → User.user_id

13. Exam → Certificate

- One exam -> Multiple certificates (1-N, one per passing candidate)

(Pto)

- Certificate.exam_id → Exam.exam_id

14. Attempt → Certificate

- One attempt -> One certificate (1-1, the specific attempt that earned it)
- Certificate.attempt_id → Attempt.attempt_id

15. User → Notification

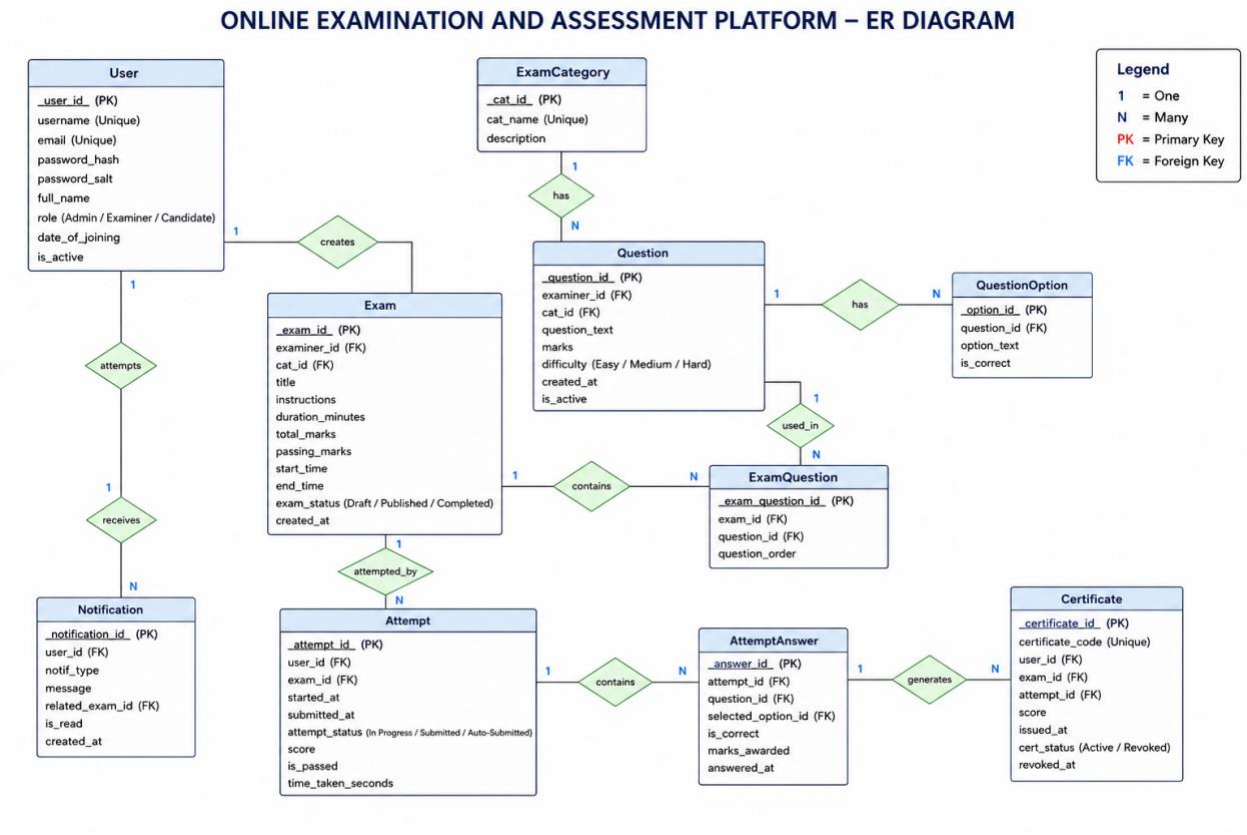
- One user -> Multiple notifications (1-N)
- Notification.user_id → User.user_id

16. Exam → Notification

- One exam -> Multiple notifications (1-N, one per recipient)
- Notification.related_exam_id → Exam.exam_id

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

IV. ER Diagram – (ai generated)



Phase 4 : Business Logic Framing (Pseudocode)

1. **Start Online Examination System**
   a. Display Main Menu
      i. User Login
      ii. Examiner Login
      iii. Admin Login
      iv. User Registration
      v. Exit
2. **User Registration**
   a. Accept user details
      i. Full Name
      ii. Email
      iii. Username
      iv. Password
   b. Validate input fields
      i. Email must be unique
      ii. Password must satisfy strength rules
   c. If validation fails
      i. Display validation error
      ii. Return to registration page
   d. If validation succeeds
      i. Hash password
      ii. Store user account in database
      iii. Display registration success message
3. **User Login**

a. Accept username/email and password
b. Validate credentials
    i. Check if account exists
    ii. Compare password hash
    iii. Check if account is active
c. If invalid
    i. Display invalid credentials message
    ii. Return to login menu
d. If valid
    i. Generate JWT token/session
    ii. Open User Dashboard

4. Examiner Login

a. Accept examiner credentials
b. Validate examiner role and credentials
c. If invalid
    i. Display access denied message
d. If valid
    i. Open Examiner Dashboard

5. Admin Login

a. Accept admin credentials
b. Validate admin role and credentials
c. If invalid
    i. Display invalid credentials message
d. If valid
    i. Open Admin Dashboard

6. User Dashboard

a. Display User Operations
    i. Browse Exams and filter exams by category
    ii. View Exam Details
    iii. Attempt Exam
    iv. View Results
    v. View Analytics Dashboard
    vi. View Certificates
    vii. View Notifications
    viii. Logout

7. Browse Exams - Allow search/filter by category

a. Fetch all published exams
b. Display exam details
    i. Exam Title
    ii. Category of exam
    iii. Duration
    iv. Passing Marks
    v. Schedule

8. Start Exam

a. User selects exam
b. Validate exam conditions
    i. Exam must be published
    ii. Current time must be within exam schedule
c. If validation succeeds :
    i. Create new attempt record
    ii. Start server-side timer
    iii. Load exam questions

9. Attempt Questions

a. Display questions one by one or as a list
b. User selects answer
c. Auto-save answer immediately
d. Allow question navigation
    i. Next , previous and change answer for any question
e. Timer should always be monitored

10. Manual Exam Submission

a. User clicks Submit Exam

b. Confirm submission action
c. If confirmed
    i. Timer should be stopped
    ii. Save final answers
    iii. Evaluate answers and calculate score accd to marking for every question
    iv. Store result
    v. Mark attempt -> Submitted

11. Resume Attempt Logic
    a. If candidate closes browser mid-exam, resume same attempt with original attempt
        i. Check

12. Timeout Logic

a. Monitor remaining time
b. If timer reaches zero
    i. Automatically save current answers
    ii. Evaluate attempt
    iii. Calculate score
    iv. Mark attempt as AutoSubmitted
    v. Store result
c. Display timeout submission message

13. Evaluate Exam

a. Fetch all submitted answers
b. For each question
    i. Compare marked option with right option
    ii. Assign marks if correct
    iii. Assign 0 if incorrect
c. Calculate total score
d. Give pass/fail status
e. Store evaluated results

14. View Results

a. Check if examiner has published the results
b. If not pulished
    i. Display “Results Not Yet Published”
c. If published
    i. Display total score, pass/fail status, time-taken other analysis per question
        - User answer
        - Correct answer
        - Marks awarded

15. View Analytics Dashboard

a. Fetch user attempt history
b. Calculate analytics
    i. Total exams attempted, Average score, Pass percentage, Best score, Certificates earned
c. Display charts/statistics

16. View Certificates

a. Fetch issued certificates
b. Display certificate details
    i. Certificate ID
    ii. Exam Name
    iii. Issued Date
c. Allow certificate download

17. View Notifications – it can be an SMTP server like EmailService

a. Fetch unread/read notifications
b. Display notifications list
c. Mark notification as read when opened

18. Examiner Dashboard

a. Display Examiner Operations
    i. Manage Questions
    ii. Manage Exams

iii. Publish Exams
iv. Publish Results
v. View Candidate Attempts
vi. Logout

19. Manage Questions

a. Create Question
   i. Enter question text
   ii. Enter options
   iii. Mark correct option
   iv. Assign marks
b. Edit Question
c. Delete Question
d. Save changes to database

20. Manage Exams

a. Create Exam
   i. Enter exam details
   ii. Set duration
   iii. Set passing marks
   iv. Assign category
   v. Add questions
b. Edit Exam
c. Delete Exam
d. Save exam changes

21. Publish Exam

a. Validate exam
   i. Must contain questions
   ii. Schedule must be valid
b. Change exam status to Published
c. Make exam visible to users

22. Publish Results

a. Fetch all attempts for exam
b. Make results visible to users
c. For each passed candidate
   i. Find their highest-scoring attempt
   ii. If that attempt's score ≥ passing marks :
      - Generate certificate
      - Store certificate record
d. Send notifications/emails to all candidates
e. Mark results as published

23. Admin Dashboard

a. Display Admin Operations
   i. Manage users, examiners, categories
   ii. View Platform Analytics
   iii. Revoke Certificates -> yet to decide BL
   iv. Logout

24. Manage Users

a. View all users
b. Activate/Deactivate users
c. Save status updates

25. Manage Examiners

a. Create examiner accounts
b. Edit examiner details
c. Deactivate examiner accounts

26. Manage Categories

a. Create category
b. Edit category
c. Delete category

27. Revoke Certificate

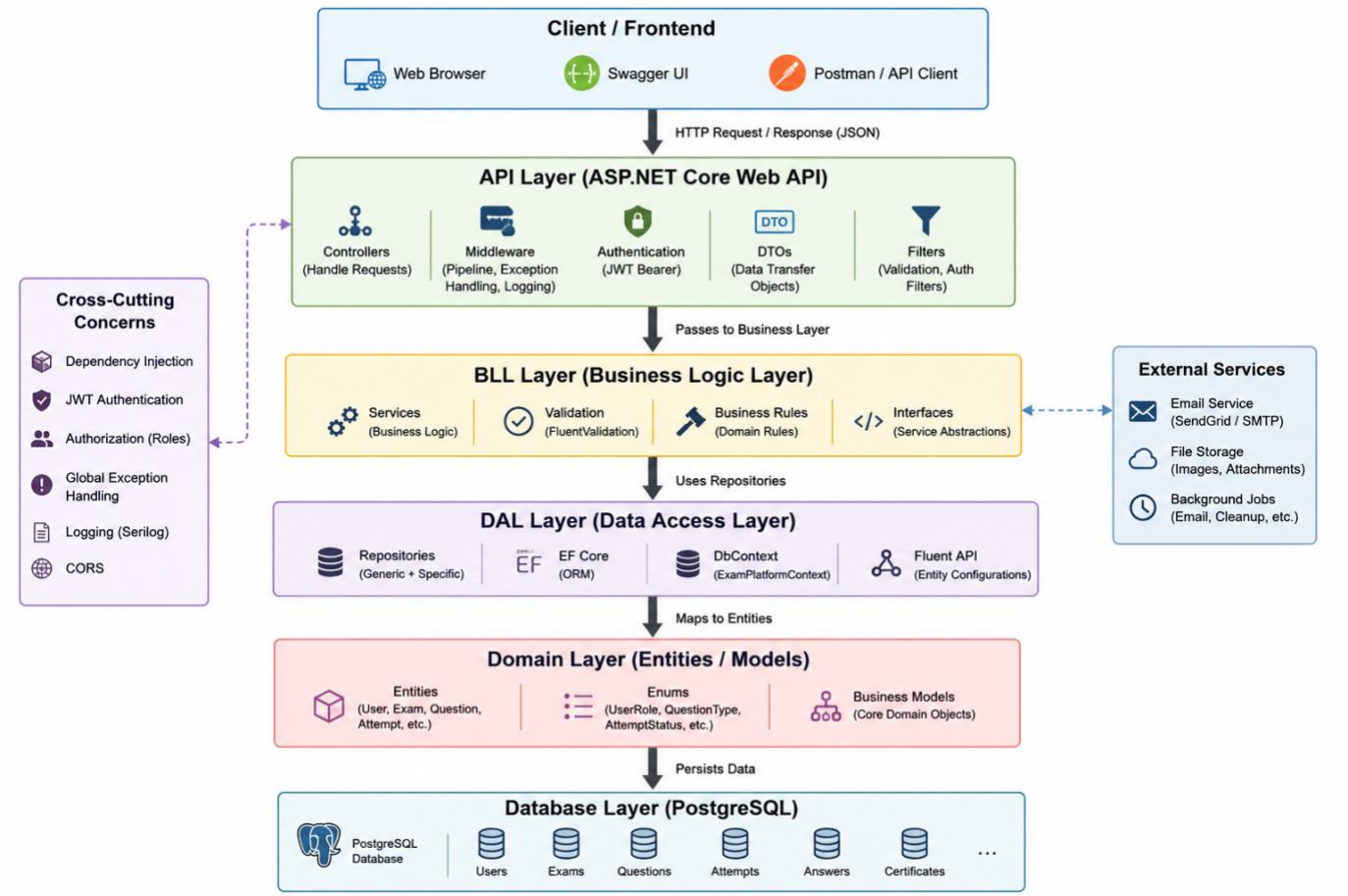
a. Select certificate
b. Change certificate status to Revoked
c. Prevent further downloads
d. Notify affected user

28. Logout

a. Destroy session / invalidate token
b. Redirect user to main menu

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

Phase 5 : Architecture Design – (yet to complete)



Solution + 4 projects + references + packages

- Domain layer (10 entities + 5 enums) ✅
- DAL layer (DbContext + 10 Fluent API configurations) ✅
- Connection string + DI wiring in API ✅
- First migration generated AND applied → DB alive ✅

GET /api/categories (any logged-in user — for browse/filter)

GET /api/categories/{id} (any logged-in user)

POST /api/categories (Admin only)

PUT /api/categories/{id} (Admin only)

DELETE /api/categories/{id} (Admin only — only if no exams/questions reference it)

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project

Phase 6 : Implementation

Approach I will be using – Code first approach

Name – Sai Nikitha
Email – sainikinsr@gmail.com
Date- 21st May 2026
Capstone Project